Getting Started with Google Gemini API

Today's goal • 3/3 items

Module 1
Make Your First Request with the Gemini API

Module 2
Model Selection and Parameters

Module 3
Built-in Tools and Structured Output

- From raw data to agent-ready information
  Video • 2 min
- Built-in tools and structured output fundamentals
  Reading • 10 min
- Structured output implementation
  Video • 6 min
- Activity: Intelligent information processor
  Practice Assignment • Grade: 100%
- Graded quiz: Optimize Gemini's responses
  Graded Assignment • Grade: 97.5%
- Course Reflection: Consolidating Your Gemini API Skills
  Dialogue • 15 min
- **Course wrap-up**
  Reading • 10 min

Course wrap-up

0:11 / 11:09 2x

Throughout this course, you progressed step by step from foundational concepts to practical development. You learned how AI models process information, how to securely interact with them through an API, and how to design prompts that produce meaningful responses. You also built a simple AI text generator application that demonstrates how generative AI can be integrated into real-world software solutions.

This final reading summarizes the core concepts covered in the course. It also prepares you for the next step in your learning journey, where you will explore more advanced tools and techniques for developing AI-powered applications.

The foundations of AI communication

The course began by introducing the Gemini API and explaining how developers communicate with generative AI models. Interacting with an AI model is different from interacting with traditional software systems such as databases.

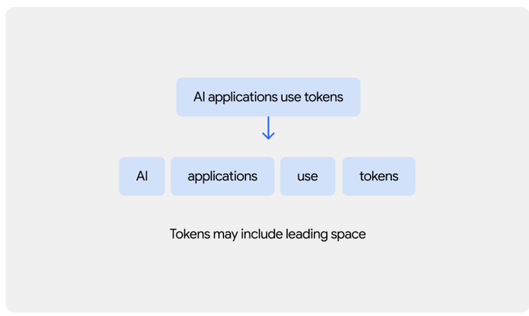
In a typical database system, a user requests a specific piece of stored information. For example, if a system searches for a particular user ID, it returns the exact record associated with that ID. The interaction is precise and predictable.

Generative AI systems operate differently. Instead of retrieving stored data, they generate new content based on patterns learned during training. When you provide a prompt to Gemini, the model analyzes the input and predicts the most likely sequence of words that should follow. This process allows the model to generate explanations, summaries, answers, or creative text.

Understanding this predictive behavior is essential for you when you are building AI-powered applications. Instead of asking for exact answers, you'll guide the model toward generating useful outputs.

One of the foundational ideas introduced in the course was the concept of tokens. Tokens are the basic units of text that the model processes. They do not always correspond directly to words. A token may represent a whole word, a part of a word, punctuation, or even spaces.

How tokens work



Tokens play a critical role in how AI models function. They affect both API usage cost and processing limits. Every prompt sent to the Gemini API consumes tokens, and the model's response also consumes tokens. Since API usage is often measured based on tokens processed, understanding token usage helps developers manage costs effectively.

Tokens are also connected to the concept of the context window. The context window represents the maximum number of tokens the model can consider during a single request.

You can think of the context window as the model's working memory. If the prompt or conversation exceeds this limit, the model may lose access to earlier parts of the input. This can affect the quality or accuracy of the generated response.

Security and your first API request

Before building applications with the Gemini API, the course emphasized the importance of security.

Accessing the Gemini API requires an API key, which acts as a credential that allows your application to communicate with Google's servers. This key identifies your project and tracks how the API is used.

Because API keys grant access to your account and usage quota, they must be protected carefully. If an API key becomes publicly visible, unauthorized users could potentially use it to send requests on your behalf.

For this reason, you should store API keys securely using environment variables rather than placing them directly inside your code. This practice prevents sensitive credentials from being accidentally exposed when sharing code or uploading projects to public repositories.

After securing the development environment, you installed the Software Development Kit (SDK) for your chosen programming language, such as Python or JavaScript.

An SDK simplifies communication with the Gemini API by providing prebuilt libraries and tools. Instead of manually creating network requests and handling authentication, you can use these tools to send prompts and receive responses with minimal effort.

Using the SDK, you created your first interaction with the Gemini model. You provided a prompt and received a generated response.

Although the process required only a few steps, this interaction demonstrated the fundamental workflow of an AI-powered application. The system collects user input, sends the request to the model through the API, and then displays the generated response.

The API response contains more than just generated text. It also includes additional information such as metadata and safety details. You must extract the relevant generated content from the response before presenting it to users.

Learning how to interpret and process these responses is an important part of building functional AI applications.

Choosing the right model

After generating text successfully, the course introduced the concept of model selection.

Different AI models are designed for different purposes. Some models focus on speed and efficiency, while others prioritize reasoning ability and deeper language understanding. During the course, you explored the differences between Gemini Flash and Gemini Pro.

Gemini Flash is designed for fast responses and cost efficiency. It works well for applications that require high volumes of quick interactions, such as chatbots or automated customer support tools.

Gemini Pro is designed for more complex tasks that require deeper reasoning and more detailed analysis. Applications that involve summarizing lengthy documents, analyzing complex information, or generating structured insights benefit from the advanced capabilities of this model.

Choosing the right model depends on several factors, including speed, cost, and the complexity of the task being performed.

Feature	Gemini Pro	Gemini Flash
Primary use case	Complex analysis, multi-step reasoning, longer or more nuanced tasks	High-volume, low-latency workloads; real-time/agentic use; cost-sensitive apps
Reasoning complexity	Higher—best when deep reasoning quality matters most	Good—optimized for speed and throughput with solid reasoning
Typical latency	Moderate to higher	Low
Relative cost	Higher per-token	Lower per-token

Transparency through model thinking

Even with the correct model, you may need to understand why a model produced a specific result. You learned about model thinking, a feature that provides a summary of the model's internal reasoning steps. This transparency is invaluable for debugging and refining prompts.

By configuring the ThinkingConfig and setting a thinkingLevel (such as "low" or "high"), you can control the depth of the model's reasoning process. Reviewing these thoughts helps identify if a model made an incorrect assumption or misinterpreted a constraint, allowing you to iterate on your instructions with greater precision.

Structuring AI outputs

Another important challenge in AI development is maintaining consistency in responses.

Large language models are designed to generate natural language, which can sometimes lead to unpredictable formats. While this flexibility can be useful for conversational interactions, software systems often require structured data that can be easily processed.

To address this need, the course introduced the concept of structured output using JSON Schema.

JSON, or JavaScript Object Notation, is a widely used format for representing structured data. It organizes information into clearly defined fields and values, making it easier for computer systems to read and process.

A JSON Schema defines the structure that the model should follow when generating output. It acts as a template specifying the fields that should appear in the response and the type of information each field should contain.

For example, if an application is designed to extract information from resumes, the model can be instructed to provide structured data fields such as the candidate's name, email address, and years of experience. Instead of producing descriptive paragraphs, the model returns information organized into clearly defined data elements. This structured format allows applications to store and process the information immediately without requiring additional text analysis.

Using structured outputs transforms generative AI models into powerful tools for data extraction and automated workflows.

Key takeaways

By the end of this course, you have developed a strong foundation in working with the Google Gemini API and building basic generative AI applications. The following points summarize the most important concepts covered throughout the course:

- **Generative AI communication differs from traditional systems.** Instead of retrieving stored data, Gemini models generate responses by predicting the most likely sequence of text based on the prompt provided.
- **Tokens are the fundamental units used by AI models to process text.** Understanding tokens helps you manage both API costs and the amount of information that can be processed within a model's context window.
- **The context window determines how much information the model can consider at one time.** Managing prompt length and conversation history helps maintain accurate responses.
- **API security is essential when developing AI applications.** API keys should be stored securely using environment variables to prevent unauthorized access.
- **Software Development Kits (SDKs) simplify the process of interacting with the Gemini API.** They allow you to send prompts and receive generated responses without handling complex network requests manually.
- **Model selection is an important design decision.** Faster models such as Gemini Flash are suitable for high-volume interactions, while more advanced models like Gemini Pro are better suited for complex reasoning tasks.
- **Model thinking provides transparency.** Enabling reasoning insights allows you to see the "how" and "why" behind a model's response, which is a powerful tool for debugging.
- **Structured output enables AI-generated responses to be easily processed by applications.** Using formats such as JSON allows you to transform natural language responses into organized data.
- **Built-in tools help connect AI models to real-world information.** Features like Google Search grounding and URL context allow models to incorporate updated or external information sources.
- **Building an AI text generator demonstrates how generative AI can be integrated into practical software solutions.**

Completed